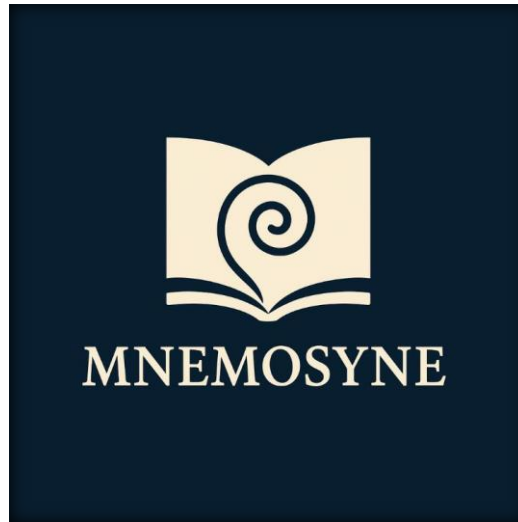


SAE 401 - DÉVELOPPEMENT D'UNE APPLICATION COMPLEXE • ÉQUIPE SATURNE A

Mnémosyne

Visualisation et suivi des cohortes étudiantes de l'IUT
via diagrammes de Sankey, alimentés par l'API ScoDoc



Soutenance du 25 juin 2026

BELQASMI Rayan • ISHAQ Faizan • RABOUDI Yacine • MERIEM Wissam • LIN HUANG Christophe • BOURG Solène

Plan de la présentation

01

Méthodologie

Choix du projet, analyse comparative du S3, axes d'amélioration et fiche de temps

02

Développement

Architecture, démonstration et présentation des ajouts techniques du S4

03

Algorithme

Calcul des flux du diagramme de Sankey

04

Travail d'équipe

Un point clé de notre collaboration

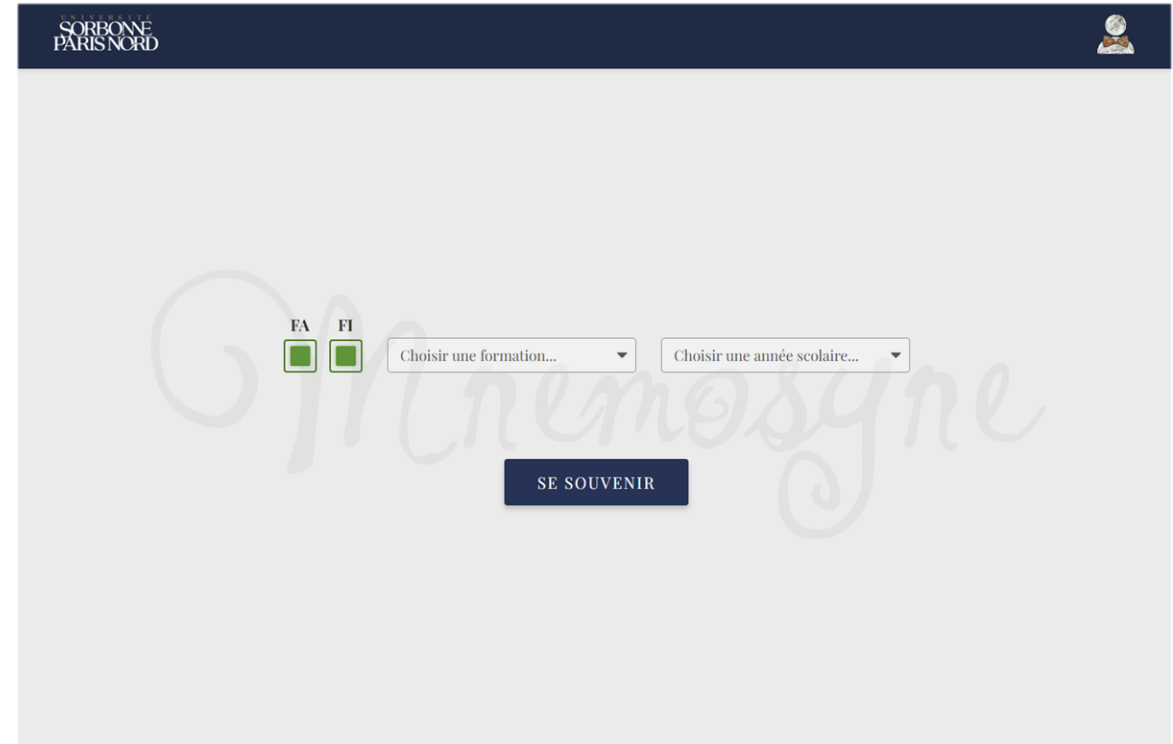
Suivre le parcours des cohortes de l'IUT

- **Le besoin** : visualiser une promotion - réussite, redoublement, abandon, réorientation.
- **La réponse** : un diagramme de Sankey interactif par formation et par année.
- **La source** : les données académiques de l'IUT via l'API ScoDoc.
- **Les acteurs** : un client en consultation et un administrateur (qui gère : règles, comptes, synchro).

Stack technique

Flask SQLite Plotly Docker pytest

Architecture en couches : Controllers → Services → Models (DAO)



Capture d'écran de la page d'accueil du site.

Les six critères retenus

1

Fiabilité & Automatisation

Collecte automatique des données plutôt qu'imports manuels (notamment avec ScoDoc).

2

Qualité de l'architecture

Séparation des responsabilités, code propre et maintenable.

3

Sécurité des données

Isolation des données sensibles, gestion des accès admin.

4

Maîtrise par l'équipe

Connaissance du code pour s'investir dans les nouveautés.

5

Potentiel d'évolution

Une base saine pour accueillir des fonctionnalités avancées.

6

Couverture du besoin

Code fidèle aux cas d'usage (cohortes, filtres, export).

Comparaison des projets candidats

Projet	Forces / Limites	Choix technique	Note
C	Fonctionnel mais PHP procédural, SQL éparpillé → forte dette technique.	Sans DAO / PHP	11/20
B	Flask + DAO avec interfaces explicites, mais dépendant de JSON figés.	Flask / DAO	14/20
A	Flask propre, schémas séparés, connexion API ScoDoc. Déploiement à renforcer.	Flask / DAO / API	16/20
E	MVC strict, procédures stockées	MVC / PHP	17/20

Tous les bons projets partagent le modèle DAO, qui isole la base de données de la logique métier. **Notre choix : le projet A.**

Notre choix : le projet A

Une décision stratégique

Maîtrise native

Projet de notre propre production : nous connaissons donc son architecture, ses forces et ses axes d'amélioration précisément.

Refus de la refonte intégrale

Reprendre un code procédural (projet C) aurait gaspillé le temps à tout réécrire au lieu de construire.

Le bon compromis

Le projet E avait la meilleure note mais le projet A offrait la base la plus saine *et* la mieux maîtrisée.

Objectif S4 : un produit sécurisé et déployable.

Quatre axes d'amélioration au S4

01

Visualisation robuste

Fiabilisation des graphiques et statistiques + tests automatisés du Sankey.

02

Authentification renforcée

Multi-administrateurs, identifiant + mot de passe, hachage bcrypt.

03

Moteur de règles refondu

Interface sans connaissance de la BDD + sécurisation anti-injection SQL.

04

Synchronisation & déploiement

Connecteur ScoDoc fiabilisé, secrets en .env, conteneurisation Docker.

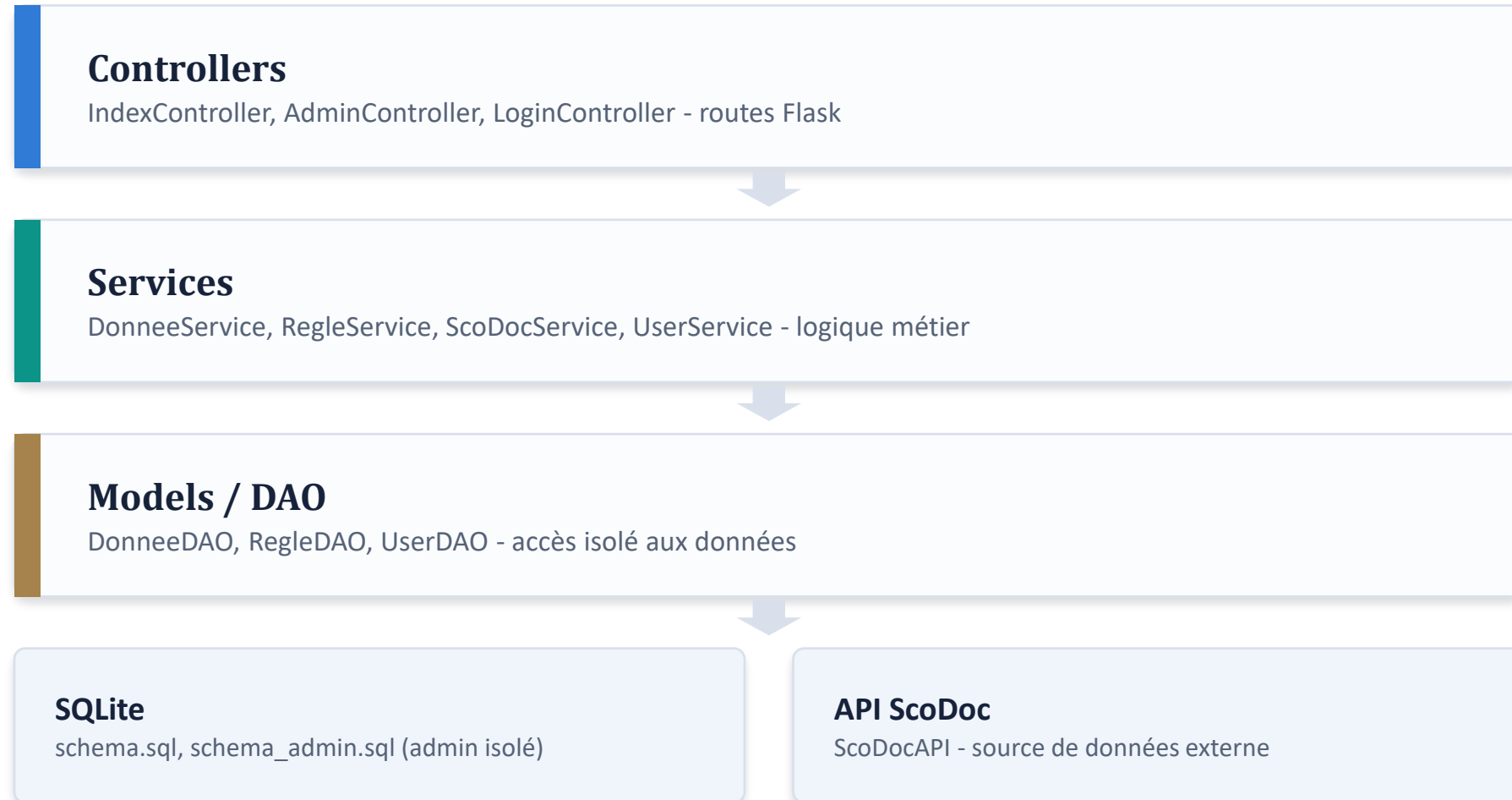
Participation de chacun

MEMBRES	TEMPS TRAVAILLÉ / PERSONNE	%
BELQASMI Rayan	16,50	19,55%
BOURG Solène	21,80	25,83%
ISHAQ Faizan	10,70	12,68%
LIN HUANG Christophe	10,50	12,44%
MERIEM Wissam	11,20	13,27%
RABOUDI Yacine	13,70	16,23%
Total général	84,40	100,00%

Capture d'écran de notre TCD pour la répartition du temps de travail.

84,4 heures-homme suivies tâche par tâche dans notre tableau de gestion et notre feuille du temps en cohérence avec les commits Git.

Architecture en couches



Deux acteurs, deux usages

Client

- Visualiser le suivi d'une cohorte (US1)
- Consulter les statistiques d'un nœud (US2)
- Filtrer par statut FI / FA / Passerelle (US3)
- Voir toutes les cohortes de l'IUT (US4)
- Exporter le diagramme en PNG / PDF (US8)

Administrateur

- S'authentifier de manière robuste (US5)
- Gérer les comptes administrateurs (US5)
- Ajouter / supprimer des règles d'analyse (US6)
- Synchroniser les données avec ScoDoc (US7)

Moteur de règles : stop à l'injection SQL

✗ S3 - condition SQL brute concaténée

```
# RegleDAO.py (S3)
def ajouter_regle(self, nom, desc, condition):
    ...
# finSQL() :
return " AND ".join(conditions)
# -> SQL concaténé = faille d'injection
```

✓ S4 - liste blanche + requête paramétrée

```
CHAMPS_AUTORISES = {"f.annee_but", ...}
OPERATEURS_AUTORISES = {"=", ">", "!=" ...}

if champ not in CHAMPS_AUTORISES:
    raise ValueError(...)
sql.append(f"{champ} {op} ?")
params.append(valeur) # jamais concaténée
```

Sécurité par conception : la valeur saisie passe par un placeholder `?` et est transmise séparément à SQLite. Le champ et l'opérateur sont validés contre une liste blanche. Cette refonte sécurise *et* rend les règles utilisables.

Une interface sans connaître la base

- **Avant** : écrire du SQL en connaissant les tables par cœur.
- **Après** : séparation logique / affichage.
- **Menu déroulant** de champs autorisés, opérateur, valeur.
- **Règles** activables / désactivables sans suppression.

Gestion des règles d'analyse

Nom	Description	Condition	Statut	Action
-----	-------------	-----------	--------	--------

Ajouter une nouvelle règle

Nom de la règle

Description

Condition

Champ

Opérateur

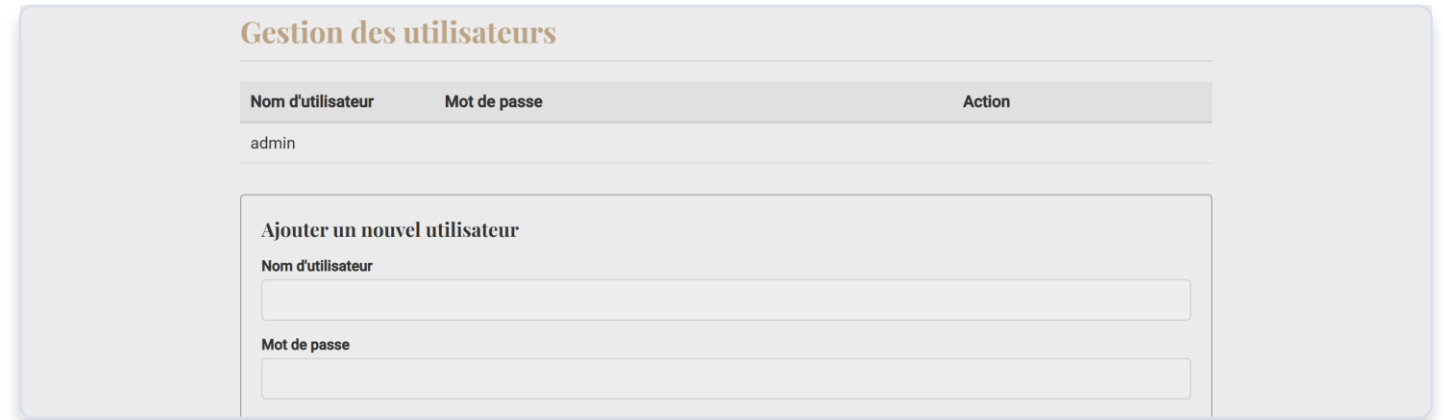
Valeur

Ajouter la règle

Capture d'écran de l'espace admin du site.

Gestion multi-administrateurs

- **S3** : un seul admin (id = 1), sans nom d'utilisateur.
- **S4** : table admin avec username UNIQUE.
- **Ajout / suppression** de comptes par l'admin.
- **Mots de passe hachés** avec bcrypt (jamais en clair).



Gestion des utilisateurs

Nom d'utilisateur	Mot de passe	Action
admin		

Ajouter un nouvel utilisateur

Nom d'utilisateur

Mot de passe

Capture d'écran de l'espace admin du site.

```
def verifyMDP(self, username, password):  
    if bcrypt.checkpw(pwd, stored_hash): return User(user)
```

Synchronisation ScoDoc fiabilisée

- **Vérification** que la BDD est initialisée avant tout import.
- **Gestion défensive** : types inattendus ignorés, exceptions remontées proprement.
- **Traçabilité** : journalisation de chaque étape de l'import.
- **Retour AJAX (JSON)** : stats affichées sans recharger la page.



```
# AdminController.py
if request.headers.get('X-Requested-With')
    == 'XMLHttpRequest':
    return jsonify({
        'stats': stats,
        'error': msg_err_import,
    })
```

Connexion & changement de mot de passe

Page de changement de mot de passe

- Nouvelle vue switchmdp.html + switchmdp.css
- Vérification de l'ancien mot de passe
- Messages d'erreur clairs pour l'utilisateur

Structuration du code

- Séparation propre HTML / CSS / JavaScript
- README complété, .gitignore (exclusion des .db)

Capture d'écran de la page de modification du mot de passe du site.

Tests unitaires automatisés

- **pytest + mocks** : on teste la logique sans toucher la vraie base (lien avec R4.02).
- **3 suites** : test_sankey, test_regle_service, test_donnee_recherche.
- **Couverture** : cas nominaux, cas limites (année absente / invalide) et résilience (BDD en panne).

```
tests/test_donnee_recherche.py::test_get_search_results_sans_annee PASSED
tests/test_donnee_recherche.py::test_get_search_results_annee_invalide PASSED
tests/test_donnee_recherche.py::test_get_search_results_succes PASSED
tests/test_regle_service.py::testajouter_regle PASSED
tests/test_regle_service.py::testmodifier_statut PASSED
tests/test_regle_service.py::test_get_regles PASSED
tests/test_regle_service.py::test_supprimer_regle PASSED
tests/test_regle_service.py::test_finSQL PASSED
tests/test_sankey.py::test_get_sankey_stats_cas_nominal PASSED
tests/test_sankey.py::test_get_sankey_stats_sans_annee PASSED
tests/test_sankey.py::test_get_sankey_stats_exception_bdd PASSED
```

Capture d'écran des tests.

✓ Exemple de test

```
# tests/test_sankey.py
def test_get_sankey_stats_sans_annee(service):
    res = service.get_sankey_stats(
        year=None, dept='INFO', ...)
    assert res is None
    service.dao.get_sankey_data
        .assert_not_called()

# garantit : pas de requête si l'année manque
```


Secrets externalisés & conteneurisation

✓ Secrets dans .env

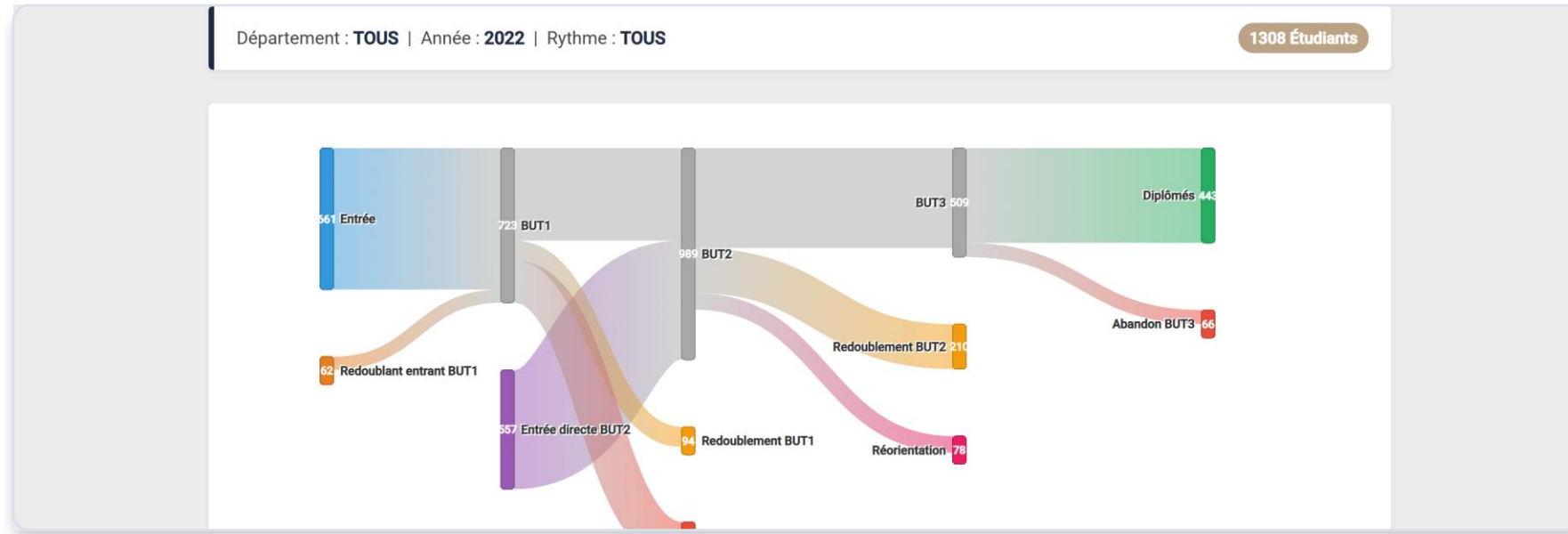
```
# app/__init__.py (S4)
app.secret_key = os.getenv("SECRET_KEY", ...)
app.config["SESSION_COOKIE_SECURE"] = \
    os.getenv("SESSION_COOKIE_SECURE")...
# clés & identifiants ScoDoc hors du code
```

✓ Docker - déploiement en une commande

```
# Dockerfile
FROM python:3.11-slim
RUN pip install -r requirements.txt gunicorn
CMD ["gunicorn", "--bind",
     "0.0.0.0:5000", "main:app"]
# serveur de production (Gunicorn)
```

On répond directement à la limite « déploiement basique » identifiée.

Le diagramme de Sankey en direct



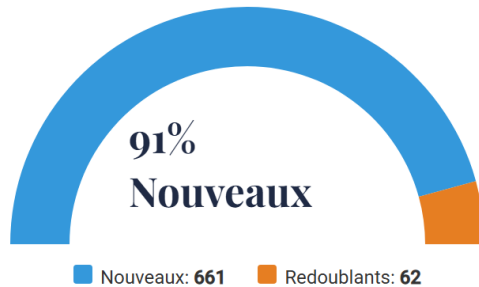
Capture d'écran de la cohorte avec TOUS les départements en 2022

- Sélection formation + année
- Flux calculés côté serveur
- Clic sur un nœud = statistiques
- Filtre FI / FA / Passerelle
- Export PNG / PDF

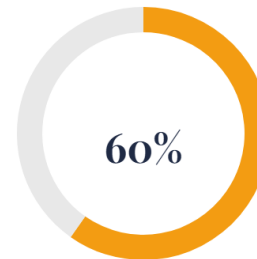
Indicateurs de performance

Indicateurs de Performance

Origine Entrée BUT 1



Passage BUT 1 → BUT 2



Passage BUT 2 → BUT 3

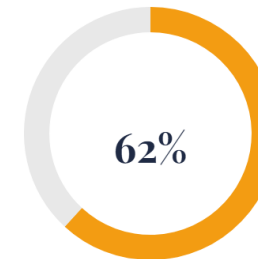


Tableau de bord généré dynamiquement à partir des données de la cohorte (jauges, doughnut Chart.js, nuage d'abandons)

91 %

de nouveaux entrants en BUT1

60–62 %

de passage BUT1→2 / BUT2→3

34 %

de diplomation globale

Calcul des flux du Sankey

- **Reconstitution** du parcours de chaque étudiant année par année (n-1 ... n+2).
- **Comparaison** des annee_but consécutives → admis, redoublant ou sortant.
- **Classification** par codes résultat (ADM, AJ, ABAN, DIP...) regroupés en ensembles.
- **Agrégation** en compteurs qui alimentent les flux du diagramme.

✓ Logique sécurisée par les tests

```
# DonneeDAO._calculate_sankey_stats()
if annee1 in p and p[annee1]['annee_but']==1:
    stats['but1_total'] += 1
    if p[annee2]['annee_but']==2:
        stats['but1_admis'] += 1
    elif p[annee2]['annee_but']==1:
        stats['but1_redouble'] += 1
    else:
        stats['but1_abandon'] += 1
```

Notre force collective : la revue de code croisée

La situation

Six développeurs sur une même base de code : risque réel de conflits Git et de régressions silencieuses sur des modules partagés (règles, Sankey).

Ce que nous en avons fait

- Découpage en blocs indépendants (visualisation, authentification, règles, synchronisation).
- Lecture et test des commits par d'autres membres avant intégration.

→ *Le point de vigilance « collaborer à 6 sur un même dépôt » est devenu notre meilleur garde-fou qualité.*

Bilan technique & personnel

Bilan technique

- Application sécurisée (anti-injection, bcrypt, secrets .env).
- Testée (pytest) et déployable (Docker).
- Synchronisation ScoDoc fiabilisée, UI des règles accessible.

Bilan personnel

- Reprendre et faire évoluer son propre code dans la durée.
- Travailler à 6 sur un dépôt Git : organisation, revue, communication.
- Adopter une démarche professionnelle : sécurité, tests, déploiement.



Merci de votre attention

Des questions ?

github.com/Solene0971/Mnemosyne

BELQASMI • ISHAQ • RABOUDI • MERIEM • LIN HUANG • BOURG

